

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 1:
The Caesar Cipher
(Classic and with a Keyword Permutation)

Elaborated:

st. gr. FAF-243 Morari Veaceslav

Verified:

asist. univ. Zaica Maia

Contents

1	ALGORITHM ANALYSIS	2
1.1	Objective	2
1.2	Tasks	2
1.3	Theoretical Notes	2
2	Introduction	3
2.1	Alphabet Encoding	3
2.2	Input Format	3
3	IMPLEMENTATION	3
3.1	Letter Encoding	3
3.2	Input Validation	4
3.3	Task 1.1: The Caesar Cipher	4
3.3.1	Algorithm Description	4
3.3.2	Implementation	4
3.3.3	Results	5
3.4	Task 1.2: The Caesar Cipher with a Permutation	5
3.4.1	Algorithm Description	5
3.4.2	Implementation	6
3.4.3	Results	6
3.5	Handling of Invalid Input	7
3.6	Comparison: Classic vs. Permutation	9
4	CONCLUSION	9
5	SOURCE CODE	9

1 ALGORITHM ANALYSIS

1.1 Objective

Study the Caesar cipher, understand why its small key space makes it insecure, strengthen it with a keyword-based permutation of the alphabet, and implement both versions for the Romanian alphabet.

1.2 Tasks

The tasks of this laboratory work are the following. First, implement the Caesar cipher for the Romanian alphabet ($n = 31$) using only the letter encoding from the assignment ($A = 0, \text{Ă} = 1, \dots, Z = 30$), with a key from 1 to 30 (Task 1.1). Second, implement the Caesar cipher with two keys: a numeric shift and a keyword of at least 7 letters that permutes the alphabet (Task 1.2). In both tasks the text is converted to uppercase, spaces are removed, and any invalid key or character must produce a clear error message and a new request for the value. Finally, draw conclusions about the security of the two ciphers.

1.3 Theoretical Notes

The **Caesar cipher** is a substitution cipher: every letter of the plaintext is replaced by the letter found a fixed number of positions k further along the alphabet. The key k is the same for encryption and decryption, $k \in \{1, 2, \dots, n-1\}$, where n is the length of the alphabet. If x and y are the numeric codes of a plaintext and of a ciphertext letter, then

$$\begin{aligned}c &= e_k(x) = (x + k) \bmod n, \\m &= d_k(y) = (y - k) \bmod n.\end{aligned}$$

When $y - k$ is negative, n is added to bring the result back into the range $0 \dots n-1$. For the Romanian alphabet, for example, with $k = 3$: $e_3(\text{Ş}) = (22 + 3) \bmod 31 = 25 = \text{U}$ and $d_3(\text{A}) = (0 - 3) \bmod 31 = 28 = \text{X}$.

The cipher has only $n - 1$ useful keys: 25 for English and 30 for Romanian. An attacker can try all of them in a fraction of a second (*brute-force attack*) and keep the only result that reads as meaningful text.

The **Caesar cipher with a permutation** uses a second key, a *keyword* k_2 , to reorder the alphabet. The keyword letters are written first, each only at its first occurrence, followed by the remaining letters in their natural order. The shift k_1 is then applied over the permuted alphabet π , where $\text{pos}(\cdot)$ is the position of a letter in π :

$$\begin{aligned}c &= \pi [(\text{pos}(x) + k_1) \bmod n], \\m &= \pi [(\text{pos}(y) - k_1) \bmod n].\end{aligned}$$

The number of key pairs (k_1, k_2) is $n! \cdot (n-1)$. For Romanian this gives $31! \cdot 30 \approx 2.47 \cdot 10^{35}$, so exhaustive search is no longer practical. However, the cipher remains *monoalphabetic*: the same plaintext letter is always replaced by the same ciphertext letter, so it can still be broken by *frequency analysis*.

2 Introduction

The Caesar cipher is one of the oldest known encryption methods. It is named after Julius Caesar, who is said to have used it with the key $k = 3$ to communicate with his generals. Today it has no practical security value, but it is the standard starting point for studying classical cryptography: it shows the idea of a secret key, modular arithmetic over an alphabet, and the difference between a large key space and real security.

Both versions in this work operate on the 31 letters of the Romanian alphabet, which includes the five diacritic letters Ă, Â, Î, Ș and Ț.

2.1 Alphabet Encoding

The letters are encoded only with the table from the assignment (Table 1). Built-in character encodings (ASCII or Unicode code points) are not used to compute the shift.

Table 1: Letter encoding of the Romanian alphabet ($n = 31$).

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

2.2 Input Format

The key (and the shift k_1 in Task 1.2) must be an integer from 1 to 30 inclusive. The text may contain only Romanian letters, in upper or lower case, and spaces; spaces are removed and the text is converted to uppercase before encryption. The keyword must contain only Romanian letters and have at least 7 letters. The letters Ș and Ț are also accepted when typed with a cedilla (Ș, Ț), so text typed on different keyboards is handled the same way.

3 IMPLEMENTATION

Both tasks are implemented in Python 3 in a single console program, `caesar_cipher.py`. A menu lets the user choose Task 1.1 or Task 1.2, then the operation (encryption or decryption), the key(s) and the text. A separate file, `test_caesar_cipher.py`, contains unit tests for the main functions.

3.1 Letter Encoding

```
1 ALPHABET = ["A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "J",
2             "K", "L", "M", "N", "O", "P", "Q", "R", "S", "Ș", "T", "Ț", "U",
3             "V", "W", "X", "Y", "Z"]
4 LOWERCASE = ["a", "ă", "â", "b", "c", "d", "e", "f", "g", "h", "i", "î", "j",
5              "k", "l", "m", "n", "o", "p", "q", "r", "s", "ș", "t", "ț", "u",
6              "v", "w", "x", "y", "z"]
7 N = len(ALPHABET) # 31
8
9 TO_UPPER = {}
10 for upper, lower in zip(ALPHABET, LOWERCASE):
11     TO_UPPER[upper] = upper
12     TO_UPPER[lower] = upper
```

```
13 TO_UPPER.update({"Ș": "S", "ș": "s", "Ț": "T", "ț": "t"})
```

Listing 1: Romanian alphabet and letter normalization

The alphabet is stored as a list in the exact order of Table 1, so the code of a letter is simply its index. The dictionary `TO_UPPER` maps every accepted character to its uppercase letter, including the cedilla variants of `Ș` and `Ț`.

3.2 Input Validation

```
1 def normalize_text(text):
2     text = unicodedata.normalize("NFC", text)
3     result = []
4     for ch in text:
5         if ch == " ":
6             continue
7         if ch not in TO_UPPER:
8             return None, ch          # first rejected character
9         result.append(TO_UPPER[ch])
10    return "".join(result), None
11
12 def parse_key(raw):
13     raw = raw.strip()
14     if not raw.isdigit():
15         return None
16     key = int(raw)
17     return key if MIN_KEY <= key <= MAX_KEY else None
```

Listing 2: Text normalization and key validation

`normalize_text` removes spaces, converts the text to uppercase and returns the first character that is not a Romanian letter, so the error message can name it. `parse_key` accepts only an integer from 1 to 30; inputs such as 0, 35, -1, 2.5 or `abc` are rejected. The functions `ask_key`, `ask_text` and `ask_keyword` repeat the question until a valid value is entered; `ask_keyword` also checks the minimum length of 7 letters.

3.3 Task 1.1: The Caesar Cipher

3.3.1 Algorithm Description

Each letter of the prepared text is replaced by its code x , the code is shifted by the key modulo 31, and the letter with the new code is written to the ciphertext. Decryption is the same operation with the key $-k$. Python's `%` operator always returns a value from 0 to $n - 1$, so $(y - k) \bmod n$ never becomes negative and no extra correction is needed.

3.3.2 Implementation

```
1 def shift_text(text, key, alphabet):
2     position = {letter: i for i, letter in enumerate(alphabet)}
3     return "".join(alphabet[(position[ch] + key) % N] for ch in text)
4
5 def caesar_encrypt(text, k):
6     return shift_text(text, k, ALPHABET)          # c = (x + k) mod n
7
8 def caesar_decrypt(text, k):
```

```
9      return shift_text(text, -k, ALPHABET)          # m = (y - k) mod n
```

Listing 3: Shift over an alphabet and the classic Caesar cipher

One function, `shift_text`, does the work for both tasks: Task 1.1 passes the normal alphabet, Task 1.2 passes the permuted one.

3.3.3 Results

Figure 1 shows the encryption of the message “Criptografie și securitate” with $k = 3$ and the decryption of the obtained ciphertext. Table 2 checks the first letters by hand.

```

Laboratory Work No. 1 - Caesar Cipher
===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 1

=== Task 1.1: Caesar cipher (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: E
Key k (1-30): 3
Message: Criptografie și securitate
  Prepared message: CRIPTOGRAFIEȘISECURITATE
  Ciphertext:      FTKSVRÎTBIKHUKTHFXTKVBVH

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 1

=== Task 1.1: Caesar cipher (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: D
Key k (1-30): 3
Ciphertext: FTKSVRÎTBIKHUKTHFXTKVBVH
  Decrypted message: CRIPTOGRAFIEȘISECURITATE

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 0
Goodbye!

C:\Users\User\Desktop\CS>

```

Figure 1: Task 1.1: encryption and decryption with $k = 3$.

Table 2: Manual check of Task 1.1 ($k = 3$).

Letter m	x	$(x + 3) \bmod 31$	Letter c
C	4	7	F
R	20	23	T
I	10	13	K
P	18	21	S
Ș	22	25	U

3.4 Task 1.2: The Caesar Cipher with a Permutation

3.4.1 Algorithm Description

The keyword is normalized like the text and must have at least 7 letters. The permuted alphabet is built by going through the keyword and then through the whole alphabet,

adding each letter only the first time it appears; the result always contains all 31 letters exactly once. The program prints the permuted alphabet under the original one, so the result can be checked by hand. Encryption and decryption use the same `shift_text` function, but over the permuted alphabet.

3.4.2 Implementation

```

1 def build_permuted_alphabet(keyword):
2     permuted = []
3     for letter in keyword + "".join(ALPHABET):
4         if letter not in permuted:
5             permuted.append(letter)
6     return permuted
7
8 def caesar2_encrypt(text, k1, keyword):
9     return shift_text(text, k1, build_permuted_alphabet(keyword))
10
11 def caesar2_decrypt(text, k1, keyword):
12     return shift_text(text, -k1, build_permuted_alphabet(keyword))

```

Listing 4: Permuted alphabet and the Caesar cipher with two keys

3.4.3 Results

For $k_1 = 5$ and $k_2 = \text{CRIPTOGRAFIE}$ the keyword contributes 10 distinct letters (the second R and the second I are dropped). Table 3 shows the resulting permuted alphabet and the substitution it produces.

Table 3: Caesar cipher with keys $k_1 = 5$ and $k_2 = \text{CRIPTOGRAFIE}$.

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
m	C	R	I	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î	J	K	L	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z
$x + k_1$	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	0	1	2	3	4
c	O	G	A	F	E	Ă	Â	B	D	H	Î	J	K	L	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z	C	R	I	P	T

Figure 2 shows the encryption of “Țara mea Moldova” and the decryption of the ciphertext YGBȚHBȚĂȘLĂCB. For example, Ț has position 24, so it becomes the letter at position $(24 + 5) \bmod 31 = 29$, which is Y; A (position 7) becomes B (position 12).

```

Laboratory Work No. 1 - Caesar Cipher

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 2

=== Task 1.2: Caesar cipher with two keys (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: E
Key 1 - shift k1 (1-30): 5
Key 2 - keyword (at least 7 Romanian letters): Criptografie
Keyword: CRIPTOGRAFIE
Original alphabet:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
Permuted alphabet:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z
Message: Țara mea Moldova
Prepared message: ȚARAMEAMOLDOVA
Ciphertext: YBGBȚHBȚĂȘLĂCB

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 2

=== Task 1.2: Caesar cipher with two keys (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: D
Key 1 - shift k1 (1-30): 5
Key 2 - keyword (at least 7 Romanian letters): criptografie
Keyword: CRIPTOGRAFIE
Original alphabet:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
Permuted alphabet:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z
Ciphertext: YBGBȚHBȚĂȘLĂCB
Decrypted message: ȚARAMEAMOLDOVA

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 0
Goodbye!

```

Figure 2: Task 1.2: permuted alphabet, encryption and decryption.

3.5 Handling of Invalid Input

Figures 3 and 4 show the messages displayed for invalid input: a key outside the range 1–30, a key that is not a number, a message with digits or punctuation, a keyword that is too short and a keyword with an invalid character. In every case the program states the allowed values and asks for the value again.

```

Laboratory Work No. 1 - Caesar Cipher

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 1

=== Task 1.1: Caesar cipher (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: E
Key k (1-30): 0
    Error: the key must be an integer from 1 to 30 inclusive.
Key k (1-30): 35
    Error: the key must be an integer from 1 to 30 inclusive.
Key k (1-30): abc
    Error: the key must be an integer from 1 to 30 inclusive.
Key k (1-30): 3
Message: Salut 2026!
    Error: invalid character '2'. Only letters of the Romanian alphabet (A-Z, Ă, Â, Î, Ș, Ț, upper or lower case) and spaces are allowed.
Message: Salut
    Prepared message: SALUT
    Ciphertext:   TBOXV

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 0
Goodbye!

C:\Users\User\Desktop\CS>

```

Figure 3: Task 1.1: messages for a wrong key and an invalid character.

```

Laboratory Work No. 1 - Caesar Cipher

1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 2

=== Task 1.2: Caesar cipher with two keys (Romanian alphabet, n = 31) ===
Operation - [E]ncrypt or [D]ecrypt: E
Key 1 - shift k1 (1-30): 42
    Error: the key must be an integer from 1 to 30 inclusive.
Key 1 - shift k1 (1-30): 5
Key 2 - keyword (at least 7 Romanian letters): cifru
    Error: the keyword must contain at least 7 letters (you entered 5).
Key 2 - keyword (at least 7 Romanian letters): cifru#2
    Error: invalid character '#' in the keyword. Only letters of the Romanian alphabet (A-Z, Ă, Â, Î, Ș, Ț) are allowed.
Key 2 - keyword (at least 7 Romanian letters): Criptografie
    Keyword: CRIPTOGRAFIE
    Original alphabet:
    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
    Permuted alphabet:
    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z
Message: Țara mea
    Prepared message: ȚARAMEA
    Ciphertext:   YGBBȚHB

===== Laboratory Work No. 1 - Caesar Cipher =====
1 - Task 1.1: Caesar cipher
2 - Task 1.2: Caesar cipher with a permutation (two keys)
0 - Exit
Choose an option: 0
Goodbye!

C:\Users\User\Desktop\CS>

```

Figure 4: Task 1.2: messages for a wrong shift, a short keyword and an invalid character.

3.6 Comparison: Classic vs. Permutation

The unit tests check text normalization, key validation, the wrap-around ($Z + 1 = A$), the permuted alphabet, and that decryption restores the message for all 30 keys in both versions; all 6 tests pass. Table 4 compares the two versions.

Table 4: Comparison of the two versions of the Caesar cipher ($n = 31$).

	Classic	With permutation
Keys	$k \in \{1, \dots, 30\}$	$k_1 \in \{1, \dots, 30\}$, keyword k_2
Number of keys	30	$31! \cdot 30 \approx 2.47 \cdot 10^{35}$
Brute-force attack	trivial	impractical
Frequency analysis	vulnerable	vulnerable

4 CONCLUSION

Within this laboratory work, the Caesar cipher and the Caesar cipher with a keyword permutation were implemented in Python for the 31-letter Romanian alphabet. Both versions encrypt and decrypt correctly, which was confirmed by unit tests for all 30 keys and by manual checks of the substitution tables.

The classic Caesar cipher is not secure. Over the Romanian alphabet it has only 30 keys, so an attacker can try every key and keep the only result that reads as Romanian text. Such an attack takes less than a second even by hand-written code.

The keyword permutation removes the brute-force problem, but not the main weakness. The key space grows to $31! \cdot 30 \approx 2.47 \cdot 10^{35}$ pairs, which cannot be searched exhaustively. However, the cipher is still a monoalphabetic substitution: each plaintext letter is always replaced by the same ciphertext letter, so letter frequencies of the language (in Romanian, E, A and I are the most common) remain visible in the ciphertext. With a long enough message, frequency analysis recovers the text without knowing the key. A large key space is therefore necessary, but not sufficient for security.

In conclusion, during this work I learned how to encode a non-English alphabet with my own table instead of ASCII or Unicode codes, how to handle modular arithmetic correctly when $y - k$ is negative, how to build a permuted alphabet from a keyword without repeating letters, and how to validate user input with clear error messages.

5 SOURCE CODE

The full source code is available on GitHub: https://github.com/veacheslavv/CS_Labs/tree/main/Lab1